

NexusOptim 1.0

C++20

CPU SIMD + CUDA

Teknik Kılavuz

Parametreyi ve gradyanı alır, parametreyi yerinde günceller.
Eğitim döngüsünü, kaybı ve tensör cebirini yönetmez.

Hedef okur: NexusTrain, NexusModel veya kendi eğitim döngüsünü yazan mühendis.

Kapsam: genel API, optimizer formülleri, zamanlayıcılar, gradyan işleme, CUDA, MatrixFlash bağdaştırıcısı ve ölçülmüş hızlar.

Bu belge kütüphane kaynaklarıyla birebir uyumludur. Örnekler `#include <nexus_optim/nexus_optim.hpp>` ile derlenir.

İçindekiler

1. Kütüphane ne yapar, ne yapmaz
2. Bir adımın içinden geçen veri
3. Tasarım kuralları
4. Derleme ve ilk program
5. Temel tipler ve parametre grupları
6. Optimizer sözleşmesi
7. SGD ailesi
8. Adaptif optimizer'lar
9. Lookahead
10. Öğrenme hızı zamanlayıcıları
11. Gradyan işleme
12. Karma kesinlik
13. CUDA yolu
14. MatrixFlash bağdaştırıcısı
15. Durum kaydı
16. Tam eğitim döngüsü
17. Hız ve kısıtlar
18. Hata durumları

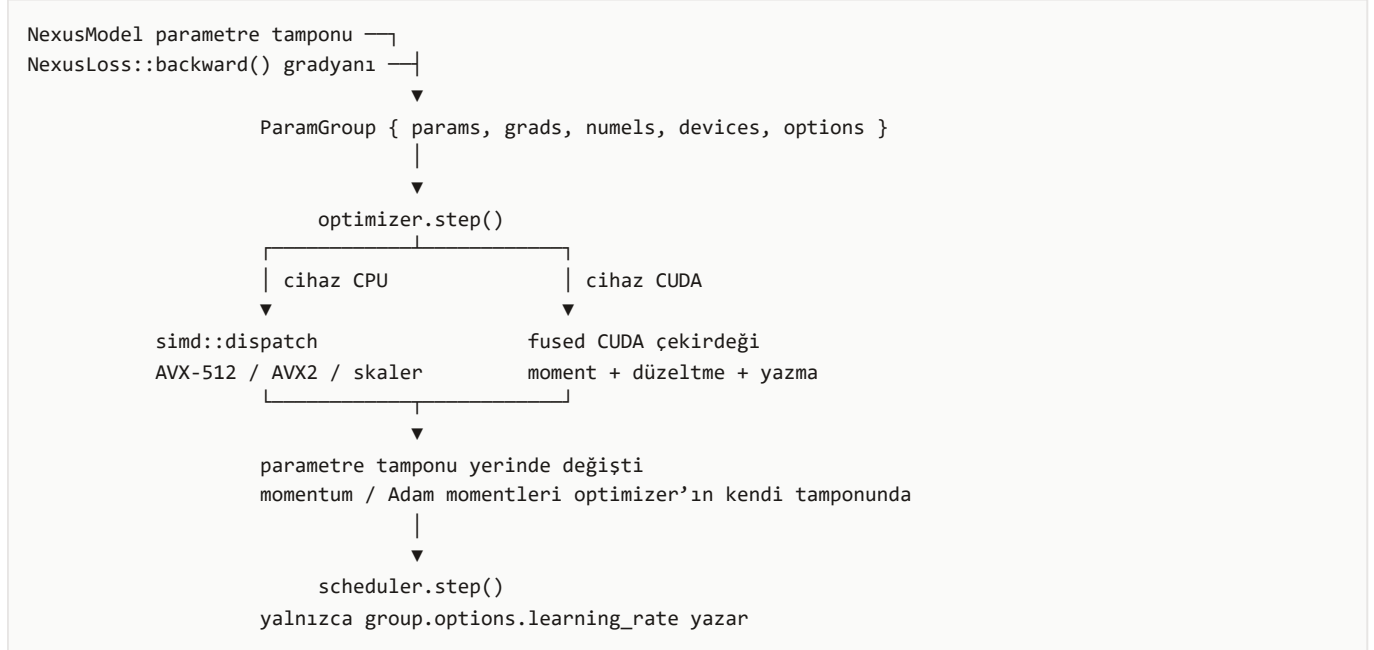
1. Kütüphane ne yapar, ne yapmaz

NexusOptim, daha büyük Nexus yığınının optimizer katmanıdır. Bir tensör girer: parametreler ve o parametrelere ait gradyanlar. `step()` parametreyi **yerinde** günceller. Geriye yeni bir tensör dönmaz.

Bileşen	İşi	NexusOptim'in onla ilişkisi
MatrixFlash Pro	Tensör depolama ve matris çekirdekleri	Parametre ve gradyan işaretçisi okunur. <code>matmul</code> çağrılmaz.
NexusModel	Katmanlar ve <code>parameters()</code>	Liste, <code>ParamGroup</code> haline getirilir.
NexusLoss	Kayıp ve geri yayılım	Gradyan tamponunu doldurur. Optimizer onu tüketir.
NexusOptim	Güncelleme, zamanlama, gradyan ölçekleme	Bu kütüphane.
NexusTrain	Epoch döngüsü, kayıt, günlük	<code>zero_grad</code> , <code>step</code> ve zamanlayıcıyı o çağırır.

Tek cümlelik sözleşme: gradyanı al, parametreyi en hızlı güvenli yolla güncelle. Kayıp değeri, batch veya epoch sayacı optimizer'ın içinde tutulmaz.

2. Bir adımın içinden geçen veri



Moment tamponları (hız, birinci ve ikinci moment, Lion tamponu) parametrenin yanında durmaz. Optimizer kurucusunda bir kez ayrılır. CPU'da 64 bayt hizalı host bellek, CUDA'da `cudaMalloc` ile cihaz belleği kullanılır. `step()` içinde `new` veya `malloc` yoktur.

3. Tasarım kuralları

- C RTP.** `OptimizerBase<Derived, ScalarT>` üzerindeki `step()` sanal değildir. `static_cast<Derived*>` ile `step_impl()` 'e iner. Sanal çağrı yalnızca zamanlayıcılarda vardır; onlar adım başına bir kez çalışır.
- Ödünç tampon.** `ParamGroup` işaretçi tutar, sahip olmaz. Çağırın, optimizer yaşadığı sürece belleği sağ tutar.
- Structure-of-arrays.** Parametre işaretçisi, gradyan işaretçisi, uzunluk ve cihaz ayrı dizilerdedir.
- Tek geçiş.** Adam ailesinde moment güncellemesi, bias düzeltmesi ve parametre yazması aynı döngüdedir. Ara tensör üretilmez.
- SIMD seçimi bir kez.** CPUID sürecin başında okunur. Sıra AVX-512, AVX2+FMA, NEON, skalerdir. Zorlamak için `force_simd_level`.

- **Paralel eşik.** 50.000 elemanın altı çağırın iş parçacığındadır. Üstü, kurucuda ısınmış iş havuzuna bölünür. LAMB ve LARS normu tensörün tamamı üzerinden hesaplanır; parçalar birbirinden bağımsız güven oranı üretmez.
- **Şablon skaler.** Varsayılan `float` . `double` CPU yolunda çalışır. CUDA çekirdekleri FP32'tir. `float16` ve `bfloat16` depolamada moment ve parametre kopyası FP32 tutulur, adım sonunda dar tipe yazılır.

4. Derleme ve ilk program

```
cmake -S . -B build
cmake --build build --config Release
build\tests\Release\nexus_optim_tests.exe
```

Seçenek	Varsayılan	Anlam
NEXUS_OPTIM_WITH_CUDA	OFF	Fused CUDA çekirdekleri. RTX 3070 için mimari 86.
NEXUS_OPTIM_ENABLE_AVX512	ON	Birim derlenir. CPU yoksa çalışma anında AVX2'ye düşülür.
NEXUS_OPTIM_BUILD_TESTS	ON	GoogleTest.
NEXUS_MATRIXFLASH_INCLUDE	boş	MatrixFlash <code>include/</code> yolu.

```
cmake -S . -B build-cuda -DNEXUS_OPTIM_WITH_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build-cuda --config Release
```

C++20 zorunludur. Kütüphane hedefinde uyarılar hatadır (`/W4` `/WX` veya `-Wall` `-Wextra` `-Werror`). AVX içsellersi yalnızca kendi çeviri birimindedir; ana kütüphane AVX'siz CPU'da çökmez.

En kısa çalışan örnek

```
#include <nexus_optim/nexus_optim.hpp>
#include <iostream>
#include <vector>

int main() {
    std::vector<float> p{1.f, 2.f, 3.f};
    std::vector<float> g{0.1f, -0.2f, 0.5f};

    nexus_optim::ParamGroupOptions group;
    group.learning_rate = 0.1;

    nexus_optim::SGD<float> opt(
        {nexus_optim::make_group(p.data(), g.data(), p.size(), group)}, {});

    opt.zero_grad();          // bu örnekte g'yi sıfırlar; sonra tekrar doldurun
    g = {0.1f, -0.2f, 0.5f};
    opt.step();               // p := p - 0.1 * g

    for (float v : p) std::cout << v << ' ';
}
```

Beklenen çıktı: `0.99 2.02 2.95` .

5. Temel tipler ve parametre grupları

Device ve SimdLevel

`Device::CPU` ve `Device::CUDA` . `make_group` tek tensör için cihazı CPU kabul eder. CUDA tensörünü elle kurun:

```
nexus_optim::ParamGroup<float> group;
group.params = {device_params};
group.grads = {device_grads};
group.numels = {n};
group.devices = {nexus_optim::Device::CUDA};
group.options.learning_rate = 1e-3;
```

`detected_simd_level()` sürecin gördüğü yolu döner. `force_simd_level(SimdLevel::Scalar)` kıyas için skaler yolu zorlar. `std::nullopt` algılanan yola döner. Desteklenmeyen bir yolu zorlamak `false` döner ve mevcut seçimi değiştirmez. `simd_level_name` ve `version()` tanı dizgileridir. Sürüm metni `"1.0.0"`.

ParamGroupOptions

Alan	Varsayılan	Kim okur
<code>learning_rate</code>	1e-3	Her <code>step()</code> ve zamanlayıcı. Zamanlayıcı bu alanı yerinde yazar.
<code>weight_decay</code>	0	Yalnızca <code>weight_decay_override == true</code> iken.
<code>weight_decay_override</code>	false	false ise optimizer <code>Options::weight_decay</code> kullanılır.

Transformer pratiği: gövde grubunda decay optimizer seçeneğinden gelir. Bias ve LayerNorm grubunda `weight_decay_override = true` ve `weight_decay = 0` yazın. Açık sıfır, "alanı doldurmadım" sanılıp optimizer değerine düşmesin diye bu bayrak vardır.

make_group

```
template <typename ScalarT>
ParamGroup<ScalarT> make_group(ScalarT* params, ScalarT* grads,
                               std::size_t numel, ParamGroupOptions options = {});

template <typename ScalarT>
ParamGroup<ScalarT> make_group(std::vector<ScalarT*> params,
                               std::vector<ScalarT*> grads,
                               std::vector<std::size_t> numels,
                               ParamGroupOptions options = {},
                               Device device = Device::CPU);
```

`validate()` dört dizinin uzunluğunu eşit ister. `numel > 0` iken işaretçi `nullptr` olamaz. Uzunluğu sıfır olan bir tensör grubun içinde durabilir ve adımda atlanır. Grupsuz bir optimizer kurmak `std::invalid_argument` fırlatır.

6. Optimizer sözleşmesi

Bütün somut optimizer'lar `OptimizerBase` üzerinden şu yüzeyi paylaşır. Şablon argümanı yazılmazsa skaler `float` olur: `SGD<>` ile `SGD<float>` aynı tiptir.

Üye	Sözleşme
<code>step()</code>	Bir güncelleme. Ayırma yapmaz. Sanal değildir.
<code>zero_grad(bool set_to_none = true)</code>	Ödünç gradyanı sıfırlar. İşaretçiler kütüphanenin olmadığı için PyTorch'taki "None yap" burada da belleği sıfırlar. Bayrak çağrı uyumu için durur.
<code>state_dict()</code> / <code>load_state_dict</code>	Adım sayacı, grup öğrenme hızları ve moment baytları.
<code>param_groups()</code>	Zamanlayıcının yazdığı vektör. Bu vektörü yeniden boyutlandırmayın; zamanlayıcı öğelerin adresini tutar.
<code>step_count()</code>	Tamamlanmış güncelleme sayısı. İlk <code>step()</code> sonrası 1.

Kopya kurucu yoktur. Taşıma vardır. İç optimizer'ı `Lookahead` sarmalına verirken `std::move` kullanın.

Öğrenme hızının kaynağı **grup alanıdır**, `Options::lr` tek başına adımı yönetmez. İkisini de aynı değere ayarlayın. `Options::lr` bazı formüllerde (RADam dikdörtgen katsayısı, Adagrad sönümü) grup hızıyla birlikte okunur; grup hızı zamanlayıcı tarafından değiştiğinde adım o anki grup değerini kullanır.

7. SGD ailesi

Üçü aynı çekirdeği paylaşır. `SgdVariant` kurulurken momentum ve Nesterov bayrağını seçer.

```
g ← g + λ p (λ ≠ 0)
v ← μ v + (1 - τ) g (μ ≠ 0, τ sönüm / dampening, varsayılan 0)
Nesterov ise g ← g + μ v, değilse g ← v
p ← p - γ g
```

Sınıf	Varsayılanlar
<code>SGD<T></code>	lr 1e-3, momentum 0, weight_decay 0, dampening 0, nesterov false
<code>MomentumSGD<T></code>	momentum 0.9
<code>NesterovSGD<T></code>	momentum 0.9, nesterov true

```
using nexus_optim::MomentumSGD;
MomentumSGD<float>::Options opt;
opt.lr = 0.1;
opt.momentum = 0.9;

nexus_optim::ParamGroupOptions pg;
pg.learning_rate = 0.1;

MomentumSGD<float> sgd({nexus_optim::make_group(p, g, n, pg)}, opt);
sgd.step();
```

Düz SGD'de hız tamponu ayrılmaz. Momentum ve Nesterov'da her tensör için bir hız tamponu vardır, başlangıcı sıfırdır. İlk adımda $v = g$ olduğundan Nesterov güncellemesi $g + \mu g$ olur. Tek eleman, lr 0.1, μ 0.9, g 0.5, p 1 için sonuç 0.905'tir.

8. Adaptif optimizier'lar

Hepsi için `Options::lr` ile `group.learning_rate` birlikte kurulur. Formüller PyTorch 2.4 tek tensör yoluna karşılık gelir. Bias düzeltmesi adım sayısına göre bir kez hesaplanır, eleman döngüsüne girmez.

Adagrad

```
γt = γ / (1 + (t - 1) · lr_decay)
s ← s + g2 (g içine λp eklenmiş hali)
p ← p - γt g / (√s + ε)
```

Varsayılan: lr 1e-2, lr_decay 0, eps 1e-10. Durum, kümülatif kare toplamıdır ve sıfırdan büyür.

```
nexus_optim::Adagrad<float>::Options o;
o.lr = 0.1; o.eps = 1e-8;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 0.1;
nexus_optim::Adagrad<float> ada({nexus_optim::make_group(p, g, n, pg)}, o);
```

RMSProp

```
v ← α v + (1 - α) g2
ortalama = √v + ε (centered ise √(v - g̃2) + ε)
momentum varsa b ← μ b + g / ortalama, p ← p - γ b
yoksa p ← p - γ g / ortalama
```

Varsayılan: lr 1e-2, alpha 0.99, eps 1e-8, momentum 0, centered false. Momentum veya centered açıksa ek tampon kurulur.

Adadelata

```
E[g2] ← ρ E[g2] + (1 - ρ) g2
Δ ← g · √(E[Δ2] + ε) / √(E[g2] + ε)
E[Δ2] ← ρ E[Δ2] + (1 - ρ) Δ2
p ← p - γ Δ
```

Varsayılan lr 1'dir. Öğrenme hızı ölçek görevi görür; algoritma kendisi bir adım büyüklüğü üretir. ρ varsayılan 0.9, ε 1e-6.

Adam ve AdamW

İki sınıf aynı `AdamImpl` şablonunun iki örneğidir. AdamW kopya değildir; `decoupled` bayrağı weight decay'in momentten önce mi sonra mı uygulandığını seçer.

Adam (eşleşmiş sönüm): $g \leftarrow g + \lambda p$, sonra momentler.
AdamW (ayrık sönüm): $p \leftarrow p - \gamma \lambda p$, momentler ham g'yi görür.

```
m ← β1 m + (1 - β1) g
v ← β2 v + (1 - β2) g2
m̂ = m / (1 - β1t), v̂ = v / (1 - β2t)
p ← p - γ m̂ / (√v̂ + ε)
AMSGrad açıksa payda max(v) üzerinden alınır.
```

Alan	Adam	AdamW
lr	1e-3	1e-3
beta1 / beta2	0.9 / 0.999	aynı
eps	1e-8	1e-8
weight_decay	0, gradyana eklenir	0, parametreden düşülür
amsgrad	false	false

```
nexus_optim::AdamW<float>::Options opt;
opt.lr = 1e-3;
opt.beta1 = 0.9;
opt.beta2 = 0.999;
opt.weight_decay = 0.01;

nexus_optim::ParamGroupOptions body;
body.learning_rate = 1e-3;           // decay, Options'tan

nexus_optim::ParamGroupOptions norm;
norm.learning_rate = 1e-3;
norm.weight_decay = 0.0;
norm.weight_decay_override = true;  // LayerNorm / bias

nexus_optim::AdamW<float> opti({body_group, norm_group}, opt);
```

Kapalı form, $t = 1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$, $\gamma = 1e-3$, $g = 0.1$, $p = 1$, decay yok: $\hat{m} = g$ ve $\hat{v} = g^2$ olduğundan adım yaklaşık $\gamma \cdot \text{sign}(g)$ olur ve $p = 0.999$ kalır. AdamW $\lambda = 0.01$ ile önce $p \leftarrow 1 - \gamma\lambda p = 0.99999$, sonra aynı adım, sonuç 0.99899.

NAdam

$\mu_t = \beta_1 (1 - \frac{1}{2} \cdot 0.96^t \psi)$, $\psi = \text{momentum_decay} = 4e-3$
 μ ürünü adımlar boyunca çarpılır (başlangıç 1).
 m ve v standart Adam güncellemesi.
 $p \leftarrow p + [\text{adım_g} \cdot g + \text{adım_m} \cdot m] / (\sqrt{(v / (1 - \beta_2^t))} + \epsilon)$

Varsayılan $\text{lr} = 2e-3$. `decoupled_weight_decay` false ise NAdam, true ise NAdamW. μ ürünü optimizier genelinde tek skalerdir; bütün tensörler aynı adım sayacını paylaşır.

RAdam

$\rho^\infty = 2 / (1 - \beta_2) - 1$
 $\rho_t = \rho^\infty - 2 t \beta_2^t / (1 - \beta_2^t)$
 $\rho_t > 5$ ise düzeltilmiş adaptif adım,
değilse $p \leftarrow p - \gamma m / (1 - \beta_1^t)$

İlk adımlarda varyans tahmini güvenilir olmadığı için güncelleme bias düzeltmeli SGD'dir. `decoupled_weight_decay` varsayılan false.

AdaBelief

$m \leftarrow \beta_1 m + (1 - \beta_1) g$
 $s \leftarrow \beta_2 s + (1 - \beta_2) (g - m)^2 + \epsilon$
 $p \leftarrow p - \gamma \hat{m} / (\sqrt{\hat{s}} + \epsilon)$

İkinci moment gradyanın kendisine değil, öngörülen ortalamadan sapmasına bakar. ϵ hem paydaya hem de s güncellemesinin içine eklenir. Ayrık weight decay varsayılan açıktır.

LAMB

Adam adımı $u_0 = \hat{m} / (\sqrt{\hat{v}} + \epsilon)$
 $u = u_0 + \lambda p$
 $\text{güven} = \|p\| / \|u\|$ (ikisinden biri 0 ise 1)
 $p \leftarrow p - \gamma \cdot \text{güven} \cdot u$

Güven oranı katman tensörünün tamamına aittir. Scratch tamponu kurucuda ayrılır. Büyük batch için tipik $\text{lr} = 1e-3$, $\epsilon = 1e-6$. CUDA'da norm, blok içi warp toplamı ve ardından tek `atomicAdd` ile birikir; eleman başına atomik yoktur.

LARS

$\text{güven} = \eta \|p\| / (\|g\| + \lambda \|p\| + \epsilon)$
 $\text{ölçek} = g + \lambda p$
 $v \leftarrow \mu v + \text{ölçek}$ ($\mu \neq 0$)
 $p \leftarrow p - \gamma \cdot \text{güven} \cdot v$

Varsayılan $\text{lr} = 1$, momentum 0.9, eta $1e-3$, eps $1e-8$. Momentum sıfır hız tamponu ayrılmaz.

Lion

```
c =  $\beta_1$  m + (1 -  $\beta_1$ ) g
p  $\leftarrow$  (1 -  $\gamma$   $\lambda$ ) p -  $\gamma$  · sign(c)
m  $\leftarrow$   $\beta_2$  m + (1 -  $\beta_2$ ) g
```

Tek tampon. Varsayılan lr 1e-4, β_1 0.9, β_2 0.99. sign(0) = 0. Decay her zaman ayrık uygulanır. $\lambda = 0$ ve $g > 0$, $p = 0.5$, $\gamma = 1e-3$ iken sonuç 0.499'dur.

9. Lookahead

Her k iç adımda bir:
yavaş $\leftarrow$ (1 - α) yavaş + α hızlı
hızlı $\leftarrow$ yavaş

Sarmal, iç optimizier'in türünü şablon olarak alır. Kurucuda parametrelerin kopyası yavaş ağırlık olarak alınır. $k \geq 1$ ve $\alpha \in (0, 1]$ olmalıdır.

```
nexus_optim::SGD<float> inner({group}, sgd_options);
nexus_optim::Lookahead<nexus_optim::SGD<float>> wrapped(
    std::move(inner), /*k=*/5, /*alpha=*/0.5f);

wrapped.zero_grad();
wrapped.step();           // iç step, sonra gerekirse yavaş karışım
wrapped.base();           // iç optimizier
wrapped.param_groups();   // zamanlayıcı bunu da kullanabilir
```

$k = 1$, $\alpha = 0.5$, SGD $\gamma = 0.1$, $p = 1$, $g = 1$: hızlı ağırlık önce 0.9 olur, yavaş 1 ile karışır, parametre 0.95'e çekilir.

10. Öğrenme hızı zamanlayıcıları

LRSchedulerBase::step sanaldır. Kurucu, optimizier'daki grupların learning_rate adreslerini kaydeder ve o anki değerleri baz hız sayar. Grup vektörünü büyüten bir push_back bu adresleri düşürür. Eğitim boyunca grup sayısı sabit kalmalıdır.

last_epoch -1'den başlar. İlk step() onu 0 yapar. Kosinüs ve StepLR bu yüzden ilk çağrıda baz hızı yakın durur.

Sınıf	Kurucu	Formül
StepLR	(opt, step_size, gamma = 0.1)	$\text{baz} \cdot \gamma^{\lfloor \text{epoch} / \text{step_size} \rfloor}$. step_size 2 ise düşüş üçüncü adımda.
ExponentialLR	(opt, gamma)	$\text{baz} \cdot \gamma^{\text{epoch}}$
CosineAnnealingLR	(opt, T_max, eta_min = 0)	$\eta_{\min} + (\text{baz} - \eta_{\min}) \cdot (1 + \cos(\pi t / T_{\max})) / 2$
CosineAnnealingWarmRestarts	(opt, T_0, T_mult = 1, eta_min = 0)	T_{cur}, T_i 'ye varınca sıfırlanır ve $T_i \leftarrow T_i \cdot T_{\text{mult}}$
OneCycleLR	(opt, max_lr, total_steps, pct_start = 0.3, div_factor = 25, final_div_factor = 1e4)	max/div'den max'a doğrusal, sonra kosinüsle max/(div·final)'a
LinearWarmup	(opt, warmup_steps)	$\text{baz} \cdot \min(1, (\text{epoch} + 1) / \text{warmup})$
ReduceLROnPlateau	(opt, factor = 0.1, patience = 10, threshold = 1e-4, cooldown = 0, min_lr = 0)	step(metric) zorunlu. Daha küçük metrik iyidir. İyileşme yoksa factor ile çarpılır.

```
nexus_optim::AdamW<float> opt({group}, adamw);
nexus_optim::CosineAnnealingLR cosine(opt, /*T_max=*/1000, /*eta_min=*/1e-6);
nexus_optim::LinearWarmup warmup(opt, /*warmup_steps=*/100);

for (int step = 0; step < 100; ++step) {
    opt.step();
    warmup.step();           // ilk yüz adım
}
for (int step = 100; step < 1100; ++step) {
    opt.step();
    cosine.step();
}
}
```

İki zamanlayıcıyı aynı grup üzerinde aynı adımda peş peşe koşturmak, ikisinin de “baz”ı kurucu anındaki hıza kilitlenmesi yüzünden birbiriyle kavga eder. Isınma bitince o zamanlayıcıyı bırakın ve kosinüsü, ısınmış hız kurucu bazınız olacak şekilde yeni bir nesne olarak açın. Plateau dışındaki sınıflar `step()` argümanını yoksayar. Plateau metrik almadan çağırırsa `invalid_argument` fırlatır.

`get_last_lr()` birinci grubun son hızıdır. `get_last_lrs()` hepsini döner. `OneCycle`, grupların birbiriyle oranını birinci grubun bazına bölerek korur.

11. Gradyan işleme

Bu fonksiyonlar optimizier adımıdan önce çağrılır. Parametreyi değil, gradyan tamponunu değiştirirler. CUDA gradyanı atlanır; kırpma bugün host yoldadır.

Global norm

```
template <typename ScalarT>
double clip_grad_norm_(std::vector<ParamGroup<ScalarT>>& groups,
                      double max_norm, double norm_type = 2.0);
```

toplam = $\|g\|_p$ (varsayılan $p = 2$, ayrıca ∞)
ölçek = $\min(1, \text{max_norm} / (\text{toplam} + 1e-6))$
 $g \leftarrow \text{ölçek} \cdot g$ yalnızca ölçek < 1 ise

Dönüş değeri kırpmadan önceki normdur. $g = (3, 4, 0)$, $\text{max_norm} = 1$ için norm 5'tir ve g yaklaşık $(0.6, 0.8, 0)$ olur. Paydadaki $1e-6$ PyTorch ile aynıdır.

Değer kırpma

```
clip_grad_value_(groups, /*clip_value=*/1.f); // her eleman [-c, c]
```

Biriktirme

Mikro-batch'ler arasında `zero_grad` çağırmayın. Gradyanlar geri yayılımın kendisiyle birikir. `step` öncesi ölçekleyin:

```
nexus_optim::scale_gradients_(opt.param_groups(), 1.0 / micro_batches);
opt.step();
opt.zero_grad();
```

Merkezeleştirme

```
gradient_centralization_(groups, /*channels=*/0);
```

`channels == 0` tensörü tek vektör sayar ve ortalamasını çıkarır. Kanallar verilirse `numel` kanallara bölünmelidir. Kalan eksen (spatial) 1 veya daha küçükse, bias'ta olduğu gibi tensöre dokunulmaz. Her kanal kendi ortalamasını kaybeder.

```
opt.zero_grad();
loss.backward(); // sizin geri yayılımınız
nexus_optim::gradient_centralization_(opt.param_groups(), /*channels=*/out_features);
auto total = nexus_optim::clip_grad_norm_(opt.param_groups(), 1.0);
opt.step();
```

12. Karma kesinlik

`float16` ve `bfloat16` `alignas(2)` depolama tipleridir. IEEE FP16 yuvarlaması ve BF16 round-to-nearest-even dönüşümleri `from_float` / `to_float` üzerindedir. CUDA yarı-kesinlik çekirdeği yoktur; dar depolama CPU'da şu düzenle çalışır:

1. Kurucuda parametreler FP32 master kopyaya yükseltilir.
2. Her adımda gradyan FP32'ye çevrilir. Bu tampon da kurucuda ayrılmıştır.
3. Momentler FP32'dir.
4. Güncelleme master üzerinde yapılır, sonuç dar tipe geri yazılır.

```
std::vector<nexus_optim::float16> p(n), g(n);
for (std::size_t i = 0; i < n; ++i) {
    p[i] = nexus_optim::float16::from_float(0.5f);
    g[i] = nexus_optim::float16::from_float(0.01f);
}
nexus_optim::AdamW<nexus_optim::float16>::Options o;
o.lr = 1e-3; o.weight_decay = 0.01;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 1e-3;
nexus_optim::AdamW<nexus_optim::float16> half_opt(
    {nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
half_opt.step();
float back = p[0].to_float();
```

13. CUDA yolu

Kütüphane `-DNEXUS_OPTIM_WITH_CUDA=ON` ile derlenmediyse `Device::CUDA` bir adım `std::runtime_error` fırlatır.

Derlendiyse şu çekirdekler vardır: SGD, Adam/AdamW, Adagrad, RMSProp, Adadelta, NAdam, RAdam, AdaBelief, LAMB, LARS, Lion.

- Her algoritma tek çekirdekte (LAMB ve LARS'ta norm + uygulama olarak iki sıkı çekirdekte) biter. Ayrı elementwise çağrılar yoktur.
- İş parçacıkları ardışık adresleri okur. Büyük tensörde SGD, Adam ve Lion `float4` yükler; son üç eleman skaler kuyrukla biter.
- Izgara, SM sayısının 32 katına kadar çıkar ve grid-stride ile tüm uzunluğu tarar.
- Dört non-blocking stream vardır. Her `step` kullanılan stream'i senkronize eder; çağrı döndüğünde yazma görünürdür. Küçük modellerde bu senkron, AVX2'den yavaş olabilir.
- Momentler kurucuda `cudaMalloc` ile sıfırlanır. Parametre ve gradyan işaretçileri çağıranıdır ve cihaz belleği olmalıdır.
- LAMB/LARS normu blok toplamından bir `atomicAdd(double)` ile birikir. Aynı GPU ve aynı izgara düzeninde sonuç, 2e-4 bağıl toleransla CPU'ya uyar. Atomik toplama sırası bit-tam deterministik değildir.

```
float* d_param = nullptr;
float* d_grad = nullptr;
cudaMalloc(&d_param, n * sizeof(float));
cudaMalloc(&d_grad, n * sizeof(float));
cudaMemcpy(d_param, host_p.data(), n * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_grad, host_g.data(), n * sizeof(float), cudaMemcpyHostToDevice);

nexus_optim::ParamGroup<float> g;
g.params = {d_param};
g.grads = {d_grad};
g.numels = {n};
g.devices = {nexus_optim::Device::CUDA};
g.options.learning_rate = 1e-3;

nexus_optim::AdamW<float>::Options o;
o.lr = 1e-3; o.weight_decay = 0.01;
nexus_optim::AdamW<float> gpu({g}, o);
gpu.step();
cudaMemcpy(host_p.data(), d_param, n * sizeof(float), cudaMemcpyDeviceToHost);
```

RTX 3070, sm_86, 1e6 parametre AdamW: CPU AVX2 769 μ s/adım, GPU 1168 μ s/adım. 7'den 10007'ye kadar boylarda SGD, iki adımlı AdamW, Lion ve LAMB, CPU ile $2e-4$ içinde örtüşür.

14. MatrixFlash bağdaştırıcısı

Başlık opt-in'dir. Şemsiye include onu çekmez, çünkü `matrix_pro/core/tensor.hpp` yolunu ister.

```
#include <nexus_optim/adapters/matrixflash.hpp>

// matrix_pro::Tensor float'tır. size(), host_ptr(), device_data().
auto group = nexus_optim::group_from_host(param_tensors, grad_tensors, options);
// veya
auto dev = nexus_optim::group_from_device(param_tensors, grad_tensors, options);
opt.step();
nexus_optim::mark_host_stale(param_tensors);
```

`group_from_host` indirmez; host aynanın güncel olduğunu varsayar. `group_from_device` cihaz işaretçisi yoksa hata verir. Bağdaştırıcı `matmul`, indirgeme veya MatrixFlash çekirdeği çağırır. Cihaz yazmasından sonra host ayna eskir; `mark_host_stale` bunu Tensor API'sine iletir.

15. Durum kaydı

`OptimizerState` şunları taşır: algoritma adı, adım sayacı, grup öğrenme hızları, moment blob'ları, bir avuç skaler (NAdam μ ürünü gibi). NexusTrain dosya biçiminin sahibidir. NexusOptim yalnızca bu yapıyı doldurur ve geri yükler.

```
auto snap = opt.state_dict();
// snap.blobs momentlerin ham baytlarıdır. Kendi arşivinize yazın.

nexus_optim::AdamW<float> resumed({same_layout_group}, same_options);
resumed.load_state_dict(snap);
```

Durum, parametreleri değil momentleri saklar. Yüklemeden önce parametre tensörleri checkpoint'teki değerlere zaten getirilmiş olmalıdır. Tensör sayısı ve her birinin eleman sayısı, kayıttaki blob boyutlarıyla birebir aynı olmalıdır. Adam durumu AdamW'ye yüklenmez; algoritma adı kontrol edilir.

CUDA momentleri kayıta host'a indirilir, yüklemede tekrar cihaza kopyalanır. Scratch (LAMB'in geçici güncelleme vektörü) kayda girmez.

16. Tam eğitim döngüsü

Aşağıdaki program $y = 3x + 1$ doğrusunu SGD ile oturtur. NexusModel ve NexusLoss'un yerini, okunabilir olsun diye elle yazılmış bir gradyan tutar. Gerçek yığında bu dört satır `model.forward`, `loss.backward` olur.

```
#include <nexus_optim/nexus_optim.hpp>
#include <cmath>
#include <iostream>
#include <random>

int main() {
    float weight = 0.f, bias = 0.f;
    float weight_grad = 0.f, bias_grad = 0.f;

    nexus_optim::ParamGroupOptions pg;
    pg.learning_rate = 0.05;

    nexus_optim::SGD<float> opt({
        nexus_optim::make_group(&weight, &weight_grad, 1, pg),
        nexus_optim::make_group(&bias, &bias_grad, 1, pg)
    }, {});
    nexus_optim::LinearWarmup warmup(opt, 10);

    std::mt19937 rng(7);
    std::uniform_real_distribution<float> xdist(-1.f, 1.f);

    for (int step = 0; step < 200; ++step) {
        opt.zero_grad();
        float loss = 0.f;
        constexpr int batch = 32;
        for (int i = 0; i < batch; ++i) {
            float x = xdist(rng);
            float y = 3.f * x + 1.f;
            float pred = weight * x + bias;
            float err = pred - y;
            loss += err * err;
            weight_grad += err * x;
            bias_grad += err;
        }
        weight_grad /= batch;
        bias_grad /= batch;

        nexus_optim::clip_grad_norm_(opt.param_groups(), 1.0);
        opt.step();
        warmup.step();
        loss /= batch;
        if (step == 199) {
            std::cout << weight << ' ' << bias << ' ' << loss << '\n';
        }
    }
}
```

Bu döngü yaklaşık `weight ≈ 2.89`, `bias ≈ 1.00`, kayıp 0.005 altına iner. Üretimde gradyan biriktirme `scale_gradients_` ile, çok gruplu decay AdamW örneğindeki gibi, zamanlama ise ısınma bittikten sonra tek bir kosinüs nesnesiyle yapılır.

NexusTrain'in çağıracığı yüzey

```
void train_step(nexus_optim::AdamW<float>& opt,
                nexus_optim::LRSchedulerBase& sched,
                /* model, batch, loss */) {
    opt.zero_grad();
    // forward + backward, gradyanlar ParamGroup içinde birikir
    nexus_optim::clip_grad_norm_(opt.param_groups(), 1.0);
    opt.step();
    sched.step();    // Plateau ise sched.step(validation_metric)
}
```

17. Hız ve kısıtlar

Ölçüm makinesi AVX2 görür, AVX-512 çalıştırmaz. 15 yardımcı iş parçacığı. AdamW trafik modeli eleman başına 28 bayttır: parametre okuma/yazma, gradyan okuma, iki momentin okuma/yazması. SGD modeli 12 bayttır.

İş	Süre	Bant
AdamW, 1e6, skaler	1712 μ s	16.4 GB/s
AdamW, 1e6, AVX2	899 μ s	31.1 GB/s (skalere göre 1.90 $\times$)
AdamW, 8e6, AVX2	9.3 ms	24.1 GB/s
SGD, 1e6, AVX2	176 μ s	68.2 GB/s
AdamW, 1e6, RTX 3070	1168 μ s	adım sonu senkron dahil
AdamW, 1e6, aynı makine AVX2	769 μ s	çok çekirdekli

Büyük tensörde skaler ve AVX2 yolları bellek ile `sqrt / div` gecikmesine dayanır; oran daralır. 1e6 parametrelilik GPU adımı, çekirdeğin kendisinden çok kuyruk ve senkron maliyeti taşır. Milyarlarca parametrede aynı çekirdek bant genişliğine oturur. AVX-512 bu CPU'da ölçülemedi; kod derlenir, CUID izin vermezse çağrılmaz.

40 birim testi CUDA derlemesinde geçti. Kıyas, skaler formül ile SIMD'i, kapalı form SGD/Adam'ı ve küçük-orta boy GPU'yu CPU ile yan yana koyar.

18. Hata durumları

Durum	Sonuç
Boş grup listesi	<code>invalid_argument</code>
params / grads / numels / devices boyları farklı	<code>invalid_argument</code>
Dolu tensörde null işaretçi	<code>invalid_argument</code>
Sıfır uzunluklu tensör	Atlanır, hata yoktur
Sıfır gradyan, decay 0	Parametre değişmez
NaN veya Inf gradyan	PyTorch gibi yayılır; adım sessizce yutulmaz
CUDA tensörü, CUDA kapalı derleme	<code>runtime_error</code>
CUDA üzerinde <code>double</code> veya <code>float16</code>	<code>runtime_error</code> . CUDA yolu FP32.
Yanlış algoritmanın state'i	Adam/AdamW ad uyumsuzluğunda <code>runtime_error</code>
State blob boyutu tutmuyor	<code>runtime_error</code>
Lookahead $k < 1$ veya α aralık dışı	<code>invalid_argument</code>
Negatif max_norm	<code>invalid_argument</code>
Plateau'ya metrik vermemek	<code>invalid_argument</code>
Merkezileştirmede kanal tensörü bölmüyor	<code>invalid_argument</code>

Kaynak düzeni

Yol	İçerik
include/nexus_optim/nexus_optim.hpp	Tek include.
core/	CRTP taban, grup, hizalı tampon, durum, iş havuzu, yarı tipler.
algorithms/	Optimizer başlıkları. Sıcak döngü SIMD veya CUDA çağrısıdır.
scheduler/	Sanal zamanlayıcılar.
processing/	Kırpma, ölçek, merkezileştirme.
simd/	Skaler formül, AVX2, AVX-512, NEON, dağıtım.
cuda/	Fused çekirdekler ve stream yöneticisi. Yalnızca CUDA derlemesinde bağlanır.
adapters/matrixflash.hpp	Host veya cihaz işaretçisine bağlama.
examples/	basic_sgd_example ve küçük doğrusal eğitim.
benchmarks/	SGD, AdamW ve CPU/GPU süre ölçerleri.

NexusOptim 1.0 teknik kılavuzu. Formüller kütüphanedeki skaler çekirdekle aynıdır. Sayılar, bu deponun Release derlemesinin RTX 3070 ve AVX2'li Windows makinesinde alınan ölçümleridir.